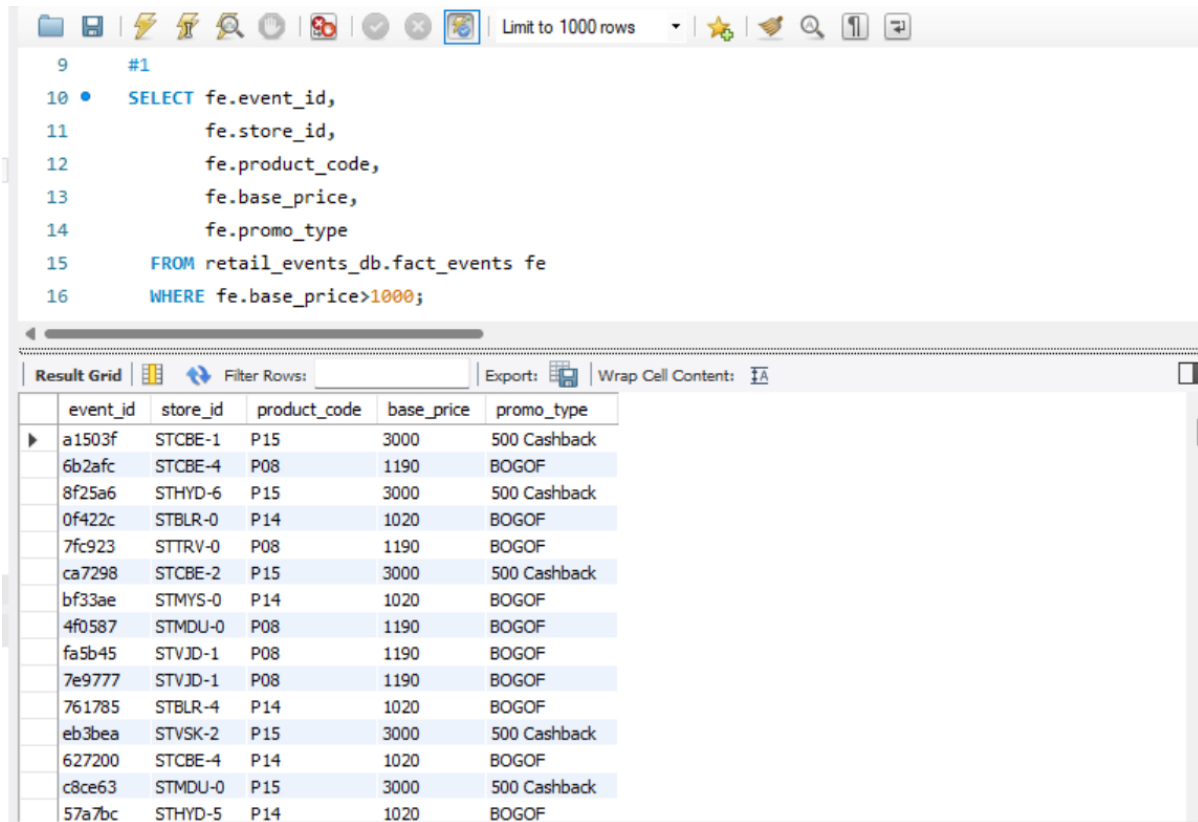


Q1. Basic Filtering – High-Value Products

Find all event records where the base_price is greater than 1,000.



The screenshot shows a SQL query editor with a toolbar at the top. The query is as follows:

```
9      #1
10     • SELECT fe.event_id,
11            fe.store_id,
12            fe.product_code,
13            fe.base_price,
14            fe.promo_type
15     FROM retail_events_db.fact_events fe
16     WHERE fe.base_price>1000;
```

Below the query editor is a 'Result Grid' showing the results of the query. The grid has columns: event_id, store_id, product_code, base_price, and promo_type. The results are as follows:

event_id	store_id	product_code	base_price	promo_type
a1503f	STCBE-1	P15	3000	500 Cashback
6b2afc	STCBE-4	P08	1190	BOGOF
8f25a6	STHYD-6	P15	3000	500 Cashback
0f422c	STBLR-0	P14	1020	BOGOF
7fc923	STTRV-0	P08	1190	BOGOF
ca7298	STCBE-2	P15	3000	500 Cashback
bf33ae	STMYS-0	P14	1020	BOGOF
4f0587	STMDU-0	P08	1190	BOGOF
fa5b45	STVJD-1	P08	1190	BOGOF
7e9777	STVJD-1	P08	1190	BOGOF
761785	STBLR-4	P14	1020	BOGOF
eb3bea	STVSK-2	P15	3000	500 Cashback
627200	STCBE-4	P14	1020	BOGOF
c8ce63	STMDU-0	P15	3000	500 Cashback
57a7bc	STHYD-5	P14	1020	BOGOF

Q2. Sorting Promotional Events

Display all events where quantity sold after the promotion was greater than 100.

```

18 #2
19 • SELECT event_id,
20         product_code,
21         promo_type,
22         `quantity_sold(before_promo)`,
23         `quantity_sold(after_promo)`
24 FROM retail_events_db.fact_events
25 WHERE `quantity_sold(after_promo)` > 100
26 ORDER BY `quantity_sold(after_promo)` DESC;
27

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

event_id	product_code	promo_type	quantity_sold(before_promo)	quantity_sold(after_promo)
e93b62	P04	BOGOF	513	2067
1a1a02	P04	BOGOF	465	2064
fe94ae	P04	BOGOF	450	1984
4eb35e	P04	BOGOF	483	1907
fe7b5d	P03	BOGOF	472	1902
0efde2	P03	BOGOF	448	1895
07e4ea	P04	BOGOF	486	1890
855f49	P04	BOGOF	481	1890
0c9f61	P03	BOGOF	433	1883
6d65d2	P04	BOGOF	480	1867
4a8c38	P04	BOGOF	469	1861
84a2f4	P04	BOGOF	423	1801
693708	P03	BOGOF	454	1788

t_events 10 x

Q3. DISTINCT Promotion Types

Find all unique promotion types used in the dataset.

```

28 #3
29 • SELECT DISTINCT promo_type
30 FROM retail_events_db.fact_events;

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

promo_type
50% OFF
25% OFF
BOGOF
500 Cashback
33% OFF

Q4. Basic Aggregation

Calculate the following for the complete fact_events table:

```
12 #4
13 • SELECT COUNT(*),
14        SUM(`quantity_sold(before_promo)`),
15        SUM(`quantity_sold(after_promo)`),
16        AVG(base_price),
17        MAX(base_price),
18        MIN(base_price)
19 FROM retail_events_db.fact_events;
```

COUNT(*)	SUM(`quantity_sold(before_promo)`)	SUM(`quantity_sold(after_promo)`)	AVG(base_price)	MAX(base_price)	MIN(base_price)
1500	209050	435473	551.9667	3000	50

Q5. Sales Volume by Promotion Type

For each promo_type, calculate:

- Number of events
- Total quantity sold before promotion
- Total quantity sold after promotion

Sort by total quantity sold after promotion in descending order.

```
41 #5
42 • SELECT promo_type,
43        COUNT(*) as total_events,
44        SUM(`quantity_sold(before_promo)` ) as total_quantity_before,
45        SUM(`quantity_sold(after_promo)` ) as total_quantity_after
46 FROM retail_events_db.fact_events
47 GROUP BY promo_type
48 ORDER BY total_quantity_after DESC;
```

Result Grid	Filter Rows:	Export:	Wrap Cell Content:
promo_type	total_events	total_quantity_before	total_quantity_after
BOGOF	500	58180	215253
33% OFF	200	63321	90576
500 Cashback	100	22299	63180
25% OFF	400	44007	38290
50% OFF	300	21243	28174

Q6. Promotion Uplift

For every promotion type, calculate:

- Total quantity before promotion
- Total quantity after promotion
- Quantity increase/decrease

The screenshot shows the MySQL Workbench interface. The SQL editor contains a query (#6) that calculates the total quantity before and after a promotion, along with the quantity change, grouped by promotion type. The query is as follows:

```
49
50 #6
51 SELECT promo_type,
52        total_quantity_before,
53        total_quantity_after,
54        total_quantity_after-total_quantity_before as quantity_change
55 FROM (SELECT promo_type,
56              SUM("quantity_sold(before_promo)") as total_quantity_before,
57              SUM("quantity_sold(after_promo)") as total_quantity_after
58        FROM retail_events_db.fact_events
59        GROUP BY promo_type) A
60 ORDER BY quantity_change DESC;
61
```

The result grid shows the following data:

promo_type	total_quantity_before	total_quantity_after	quantity_change
BOGOF	58180	215253	157073
500 Cashback	22299	63180	40881
33% OFF	63321	90576	27255
50% OFF	21243	28174	6931
25% OFF	44007	38290	-5717

Q7. Product Performance

Using fact_events and dim_products, calculate total quantity sold after promotion for every product.

The screenshot shows the MySQL Workbench interface. The SQL editor contains a query (#7) that calculates the total quantity sold after a promotion for every product. The query is as follows:

```
62 #7
63 SELECT
64     p.product_code,
65     p.product_name,
66     p.category,
67     SUM(f.`quantity_sold(after_promo)`) AS total_quantity_after
68 FROM retail_events_db.fact_events f
69 INNER JOIN retail_events_db.dim_products p
70     ON f.product_code = p.product_code
71 GROUP BY
72     p.product_code,
73     p.product_name,
74     p.category
75 ORDER BY total_quantity_after DESC;
```

The result grid shows the following data:

product_code	product_name	category	total_quantity_after
P04	Atliq_Farm_Chakki_Atta (1KG)	Grocery & Staples	81290
P03	Atliq_Sunflower_Oil (1L)	Grocery & Staples	74478
P15	Atliq_Home_Essential_8_Product_Combo	Combo1	63180
P02	Atliq_Sonamasuri_Rice (10KG)	Grocery & Staples	53235
P01	Atliq_Masoor_Dal (1KG)	Grocery & Staples	37341
P13	Atliq_High_Glo_15W_LED_Bulb	Home Appliances	29928
P14	Atliq_waterproof_Immersion_Rod	Home Appliances	23685
P07	Atliq_Curtains	Home Care	16317
P08	Atliq_Double_Redcheet_mat	Home Care	15058

Q8. Category-Level Performance

Using fact_events and dim_products, calculate for every product category:

```
76
77 #8.
78 • SELECT p.category,
79         COUNT(*) AS event_count,
80         SUM(f.`quantity_sold(before_promo)`) AS total_before,
81         SUM(f.`quantity_sold(after_promo)`) AS total_after,
82         SUM(f.`quantity_sold(after_promo)`) - SUM(f.`quantity_sold(before_promo)`) AS quantity_change
83 FROM retail_events_db.fact_events f
84 INNER JOIN retail_events_db.dim_products p
85     ON f.product_code = p.product_code
86 GROUP BY p.category
87 ORDER BY total_after DESC;
88
```

category	event_count	total_before	total_after	quantity_change
Grocery & Staples	400	126970	246344	119374
Combo1	100	22299	63180	40881
Home Appliances	200	14713	53613	38900
Home Care	400	19764	40832	21068
Personal Care	400	25304	31504	6200

Q9. Store Performance

Using fact_events and dim_stores, calculate for every city:

```
19 #9.
20 • SELECT
21     s.city,
22     COUNT(*) AS event_count,
23     SUM(f.`quantity_sold(before_promo)`) AS total_before,
24     SUM(f.`quantity_sold(after_promo)`) AS total_after
25 FROM retail_events_db.fact_events f
26 INNER JOIN retail_events_db.dim_stores s
27     ON f.store_id = s.store_id
28 GROUP BY s.city
29 ORDER BY total_after DESC;
```

city	event_count	total_before	total_after
Bengaluru	300	49171	105141
Chennai	240	39505	83273
Hyderabad	210	34363	69399
Coimbatore	150	18150	38900
Mysuru	120	18569	37470
Visakhapatnam	150	17175	33916
Madurai	120	14458	31169
Mangalore	90	7529	14929
Vijayawada	60	5297	11106
Trivandrum	60	4833	10170

Q10. Campaign Performance

Using fact_events and dim_campaigns, calculate for each campaign:

```

101 #10.
102 ● SELECT
103     c.campaign_name,
104     c.start_date,
105     c.end_date,
106     COUNT(*) AS event_count,
107     SUM(f.`quantity_sold(before_promo)`) AS total_before,
108     SUM(f.`quantity_sold(after_promo)`) AS total_after
109 FROM retail_events_db.fact_events f
110 INNER JOIN retail_events_db.dim_campaigns c
111     ON f.campaign_id = c.campaign_id
112 GROUP BY
113     c.campaign_name,
114     c.start_date,
115     c.end_date
116 ORDER BY total_after DESC;

```

	campaign_name	start_date	end_date	event_count	total_before	total_after
1	Sankranti	2024-01-10	2024-01-16	750	98731	252069
2	Diwali	2023-11-12	2023-11-18	750	110319	183404

Q11. Product Category with HAVING

Find product categories where the total quantity sold after promotion is greater than 1,000.

```

18 #Q11
19 ● SELECT
20     p.category,
21     SUM(f.`quantity_sold(after_promo)`) AS total_quantity_after,
22     AVG(f.base_price) AS average_base_price
23 FROM retail_events_db.fact_events f
24 INNER JOIN retail_events_db.dim_products p
25     ON f.product_code = p.product_code
26 GROUP BY p.category
27 HAVING SUM(f.`quantity_sold(after_promo)`) > 1000
28 ORDER BY total_quantity_after DESC;

```

```

10 #12
11 ● SELECT

```

category	total_quantity_after	average_base_price
Grocery & Staples	246344	385.0000
Combo1	63180	3000.0000
Home Appliances	53613	685.0000
Home Care	40832	490.0000
Personal Care	31504	102.3750

Q12. Store + Category Analysis

Using fact_events, dim_stores and dim_products, calculate total quantity sold after promotion for every combination of:

```
131 • SELECT
132     s.city,
133     p.category,
134     SUM(f.`quantity_sold(after_promo)`) AS total_quantity_after
135 FROM retail_events_db.fact_events f
136 INNER JOIN retail_events_db.dim_stores s
137     ON f.store_id = s.store_id
138 INNER JOIN dim_products p
139     ON f.product_code = p.product_code
140 GROUP BY
141     s.city,
142     p.category
143 ORDER BY
144     s.city,
145     total_quantity_after DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	city	category	total_quantity_after
▶	Bengaluru	Grocery & Staples	59431
	Bengaluru	Combo1	15250
	Bengaluru	Home Appliances	12909
	Bengaluru	Home Care	10006
	Bengaluru	Personal Care	7545
	Chennai	Grocery & Staples	46639
	Chennai	Combo1	12081

Q13. Promotion Effectiveness by Product

For each product, calculate:

```
#13
WITH product_sales AS (
    SELECT
        p.product_name,
        p.category,
        SUM(f.`quantity_sold(before_promo)`) AS total_before,
        SUM(f.`quantity_sold(after_promo)`) AS total_after
    FROM retail_events_db.fact_events f
    INNER JOIN retail_events_db.dim_products p
        ON f.product_code = p.product_code
    GROUP BY
        p.product_code,
        p.product_name,
        p.category
)
SELECT
    product_name,
    category,
    total_before,
    total_after,
    total_after - total_before AS quantity_change,
    ( (total_after - total_before) / NULLIF(total_before, 0) ) * 100 AS percentage_change
FROM product_sales
ORDER BY percentage_change DESC;
```

Result Grid

Filter Rows:

Export:

Wrap Cell Content:

	product_name	category	total_before	total_after	quantity_change	percentage_change
▶	Atliq_waterproof_Immersion_Rod	Home Appliances	6468	23685	17217	266.1874
	Atliq_High_Glo_15W_LED_Bulb	Home Appliances	8245	29928	21683	262.9836
	Atliq_Double_Bedsheet_set	Home Care	4203	15058	10855	258.2679
	Atliq_Curtains	Home Care	4592	16317	11725	255.3354
	Atliq_Home_Essential_8_Product_Combo	Combo1	22299	63180	40881	183.3311
	Atliq_Farm_Chakki_Atta (1KG)	Grocery & Staples	32340	81290	48950	151.3605
	Atliq_Suflower_Oil (1L)	Grocery & Staples	31309	74478	43169	137.8805
	Atliq_Masoor_Dal (1KG)	Grocery & Staples	26040	37341	11301	43.3986
	Atliq_Sonamasuri_Rice (10KG)	Grocery & Staples	37281	53235	15954	42.7939
	Atliq_Doodh_Kesar_Body_Lotion (200ML)	Personal Care	5257	7022	1765	33.5743
	Atliq_Lime_Cool_Bathing_Bar (125GM)	Personal Care	7718	10280	2562	33.1951
	Atliq_Cream_Beauty_Bathing_Soap (125...	Personal Care	6380	7697	1317	20.6426
	Atliq_Body_Milk_Nourishing_Lotion (120ML)	Personal Care	5949	6505	556	9.3461
	Atliq_Scrub_Sponge_For_Dishwash	Home Care	5762	4985	-777	-13.4849
	Atliq_Fusion_Container_Set_of_3	Home Care	5207	4472	-735	-14.1156

Q14. Campaign and Promotion Type Analysis

For each campaign and promo_type combination, calculate:

```

172 #14
173 SELECT c.campaign_name,
174        f.promo_type,
175        COUNT(*) AS event_count,
176        SUM(f.`quantity_sold(before_promo)`) AS total_before,
177        SUM(f.`quantity_sold(after_promo)`) AS total_after,
178        SUM(f.`quantity_sold(after_promo)`) - SUM(f.`quantity_sold(before_promo)`) AS quantity_change
179 FROM retail_events_db.fact_events f
180 INNER JOIN retail_events_db.dim_campaigns c
181     ON f.campaign_id = c.campaign_id
182 GROUP BY c.campaign_name, f.promo_type
183 ORDER BY c.campaign_name, quantity_change DESC;

```

campaign_name	promo_type	event_count	total_before	total_after	quantity_change
Diwali	500 Cashback	50	16791	50769	33978
Diwali	BOGOF	200	10024	34461	24437
Diwali	33% OFF	100	29152	43117	13965
Diwali	50% OFF	200	16843	22074	5231
Diwali	25% OFF	200	37509	32983	-4526
Sankranti	BOGOF	300	48156	180792	132636
Sankranti	33% OFF	100	34169	47459	13290
Sankranti	500 Cashback	50	5508	12411	6903
Sankranti	50% OFF	100	4400	6100	1700
Sankranti	25% OFF	200	6498	5307	-1191

Q15. Product Revenue Before and After Promotion

For each product, calculate:

```

85 #15
86 SELECT p.product_name,
87        p.category,
88        SUM(f.base_price * f.`quantity_sold(before_promo)`) AS revenue_before,
89        SUM(f.base_price * f.`quantity_sold(after_promo)`) AS revenue_after,
90        SUM(f.base_price * f.`quantity_sold(after_promo)`) - SUM(f.base_price * f.`quantity_sold(before_promo)`) AS revenue_difference
91 FROM retail_events_db.fact_events f
92 INNER JOIN retail_events_db.dim_products p
93      ON f.product_code = p.product_code
94 GROUP BY p.product_code, p.product_name, p.category
95 ORDER BY revenue_difference DESC;

```

product_name	category	revenue_before	revenue_after	revenue_difference
Atliq_Home_Essential_8_Product_Combo	Combo1	66897000	189540000	122643000
Atliq_Farm_Chakki_Atta (1KG)	Grocery & Staples	10851800	29100500	18248700
Atliq_waterproof_Immersion_Rod	Home Appliances	6597360	24158700	17561340
Atliq_Sonamasuri_Rice (10KG)	Grocery & Staples	32061660	45782100	13720440
Atliq_Double_Bedsheet_set	Home Care	5001570	17919020	12917450
Atliq_Sunflower_Oil (1L)	Grocery & Staples	5599512	14310708	8711196
Atliq_High_Glo_15W_LED_Bulb	Home Appliances	2885750	10474800	7589050
Atliq_Curtains	Home Care	1377600	4895100	3517500
Atliq_Masoor_Dal (1KG)	Grocery & Staples	4478880	6422652	1943772
Atliq_Doodh_Kesar_Body_Lotion (200ML)	Personal Care	998830	1334180	335350
Atliq_Time_Cool_Bathing_Bar (125GM)	Personal Care	178516	637360	458844

Q16. Classify Promotion Performance

For every promotion type, calculate total quantity before and after promotion.

	promo_type	total_before	total_after	percentage_change	performance_category
▶	BOGOF	58180	215253	269.9777	High Impact
	500 Cashback	22299	63180	183.3311	High Impact
	33% OFF	63321	90576	43.0426	Medium Impact
	50% OFF	21243	28174	32.6272	Medium Impact
	25% OFF	44007	38290	-12.9911	Low Impact

```

#16
WITH promo_sales AS (
    SELECT promo_type,
           SUM('quantity_sold(before_promo)') AS total_before,
           SUM('quantity_sold(after_promo)') AS total_after
    FROM retail_events_db.fact_events
    GROUP BY promo_type
),
promo_percentage AS (
    SELECT promo_type,
           total_before,
           total_after,
           ((total_after - total_before) / NULLIF(total_before, 0)) * 100 AS percentage_change
    FROM promo_sales
)
SELECT promo_type,
       total_before,
       total_after,
       percentage_change,
       CASE
           WHEN percentage_change >= 50
               THEN 'High Impact'
           WHEN percentage_change >= 20
               THEN 'Medium Impact'
           ELSE 'Low Impact'
       END AS performance_category
FROM promo_percentage
ORDER BY percentage_change DESC;

```

Q17. Top Products Within Each Category

Using a CTE:

Result Grid Filter Rows: Export: Wrap Cell Content:				
	category	product_name	total_quantity_after	category_rank
▶	Combo1	Atiq_Home_Essential_8_Product_Combo	63180	1
	Grocery & Staples	Atiq_Farm_Chakki_Atta (1KG)	81290	1
	Grocery & Staples	Atiq_Suflower_Oil (1L)	74478	2
	Home Appliances	Atiq_High_Glo_15W_LED_Bulb	29928	1
	Home Appliances	Atiq_waterproof_Immersion_Rod	23685	2
	Home Care	Atiq_Curtains	16317	1
	Home Care	Atiq_Double_Bedsheet_set	15058	2
	Personal Care	Atiq_Lime_Cool_Bathing_Bar (125GM)	10280	1
	Personal Care	Atiq_Cream_Beauty_Bathing_Soap (125GM)	7697	2

```
#17
WITH product_sales AS (
    SELECT p.category,
           p.product_name,
           SUM(f.`quantity_sold(after_promo)`) AS total_quantity_after
    FROM retail_events_db.fact_events f
    INNER JOIN retail_events_db.dim_products p
    ON f.product_code = p.product_code
    GROUP BY p.product_code, p.category, p.product_name
),
ranked_products AS (
    SELECT category,
           product_name,
           total_quantity_after,
           ROW_NUMBER() OVER (
               PARTITION BY category
               ORDER BY total_quantity_after DESC
           ) AS category_rank
    FROM product_sales
)
SELECT category,
       product_name,
       total_quantity_after,
       category_rank
FROM ranked_products WHERE category_rank <= 2 ORDER BY category, category_rank;
```

Q18. Best-Performing Stores Within Each City

Calculate total quantity sold after promotion for each store.

```
#18
WITH store_sales AS (
    SELECT s.city,
           s.store_id,
           SUM(f.`quantity_sold(after_promo)`) AS total_quantity_after
    FROM retail_events_db.fact_events f
    INNER JOIN retail_events_db.dim_stores s
        ON f.store_id = s.store_id
    GROUP BY s.city, s.store_id
),
ranked_stores AS (
    SELECT city,
           store_id,
           total_quantity_after,
           ROW_NUMBER() OVER (
               PARTITION BY city
               ORDER BY total_quantity_after DESC
           ) AS city_rank
    FROM store_sales
) SELECT city,
        store_id,
        total_quantity_after,
        city_rank
FROM ranked_stores
WHERE city_rank <= 2 ORDER BY city,city_rank;
```

city	store_id	total_quantity_after	city_rank
Bengaluru	STBLR-7	11866	1
Bengaluru	STBLR-6	11601	2
Chennai	STCHE-4	11547	1
Chennai	STCHE-7	11546	2
Coimbatore	STCBE-0	8657	1
Coimbatore	STCBE-2	8535	2
Hyderabad	STHYD-2	10822	1
Hyderabad	STHYD-0	10522	2
Madurai	STMDU-0	8310	1
Madurai	STMDU-2	7685	2
Mangalore	STMLR-2	5253	1
Mangalore	STMLR-1	5187	2

Q19. Campaign-Level Product Performance

For every campaign and product:

Result Grid							
Filter Rows:		Export:		Wrap Cell Content:			
	campaign_name	product_name	total_before	total_after	quantity_change	percentage_change	campaign_rank
▶	Diwali	Atliq_waterproof_immersion_Rod	2015	6950	4935	244.9132	1
	Diwali	Atliq_Curtains	2680	9214	6534	243.8060	2
	Diwali	Atliq_High_Glo_15W_LED_Bulb	3215	11053	7838	243.7947	3
	Sankranti	Atliq_Suflower_Oil (1L)	16257	61185	44928	276.3610	1
	Sankranti	Atliq_waterproof_immersion_Rod	4453	16735	12282	275.8141	2
	Sankranti	Atliq_High_Glo_15W_LED_Bulb	5030	18875	13845	275.2485	3

```
#19
WITH campaign_product_sales AS (
    SELECT c.campaign_name,
           p.product_name,
           SUM(f.`quantity_sold(before_promo)`) AS total_before,
           SUM(f.`quantity_sold(after_promo)`) AS total_after
    FROM retail_events_db.fact_events f
    INNER JOIN retail_events_db.dim_campaigns c
        ON f.campaign_id = c.campaign_id
    INNER JOIN retail_events_db.dim_products p
        ON f.product_code = p.product_code
    GROUP BY c.campaign_id, c.campaign_name, p.product_code, p.product_name
),
calculated_sales AS (
    SELECT campaign_name,
           product_name,
           total_before,
           total_after,
           total_after - total_before AS quantity_change,
           ((total_after - total_before) / NULLIF(total_before, 0)) * 100 AS percentage_change
    FROM campaign_product_sales
),
ranked_products AS (
    SELECT
        campaign_name,
        product_name,
        total_before,
        total_after,
        quantity_change,
        percentage_change,
        ROW_NUMBER() OVER (
            PARTITION BY campaign_name
            ORDER BY percentage_change DESC
        ) AS campaign_rank
    FROM calculated_sales
)
SELECT campaign_name,
       product_name,
       total_before,
       total_after,
       quantity_change,
       percentage_change,
       campaign_rank
FROM ranked_products
WHERE campaign_rank <= 3
ORDER BY campaign_name, campaign_rank;
```

Q20. Complete Promotional Performance Analysis

Create a complete analytical report at the product-category level.

product_name	category	promotional_events	total_before	total_after	quantity_change	percentage_change	revenue_before	revenue_after	revenue_change	average_base_price
Atliq_Home_Essential_8_Product_Combo	Combo1	100	22299	63180	40881	183.3311	66897000	189540000	122643000	3000.0000
Atliq_Farm_Chakki_Atta (1KG)	Grocery & Staples	100	32340	81290	48950	151.3605	10851800	29100500	18248700	330.0000
Atliq_Sonamasuri_Rice (10KG)	Grocery & Staples	100	37281	53235	15954	42.7939	32061660	45782100	13720440	860.0000
Atliq_waterproof_Immersion_Rod	Home Appliances	100	6468	23685	17217	266.1874	6597360	24158700	17561340	1020.0000
Atliq_High_Glo_15W_LED_Bulb	Home Appliances	100	8245	29928	21683	262.9836	2885750	10474800	7589050	350.0000
Atliq_Double_Bedsheet_set	Home Care	100	4203	15058	10855	258.2679	5001570	17919020	12917450	1190.0000
Atliq_Curtains	Home Care	100	16317	11725	11725	255.3354	1377600	4895100	3517500	300.0000
Atliq_Doodh_Kesar_Body_Lotion (200ML)	Personal Care	100	4592	7022	1765	33.5743	998830	1334180	335350	190.0000
Atliq_Lime_Cool_Bathing_Bar (125GM)	Personal Care	100	7718	10280	2562	33.1951	478516	637360	158844	62.0000

```

#20
) WITH product_analysis AS (
    SELECT p.product_code,
           p.product_name,
           p.category,
           COUNT(*) AS promotional_events,
           SUM(f.`quantity_sold(before_promo)`) AS total_before,
           SUM(f.`quantity_sold(after_promo)`) AS total_after,
           AVG(f.base_price) AS average_base_price,
           SUM(f.base_price * f.`quantity_sold(before_promo)`) AS revenue_before,
           SUM(f.base_price * f.`quantity_sold(after_promo)`) AS revenue_after
    FROM retail_events_db.fact_events f
    INNER JOIN retail_events_db.dim_products p
        ON f.product_code = p.product_code
    GROUP BY p.product_code, p.product_name, p.category),
    calculated_analysis AS (
        SELECT product_code,
               product_name,
               category,
               promotional_events,
               total_before,
               total_after,
               total_after - total_before AS quantity_change,
               ((total_after - total_before) / NULLIF(total_before, 0)) * 100 AS percentage_change,
               revenue_before,
               revenue_after,
               revenue_after - revenue_before AS revenue_change,
               average_base_price
        FROM product_analysis
    ),
    ranked_products AS (
        SELECT *,
               ROW_NUMBER() OVER (
                   PARTITION BY category
                   ORDER BY revenue_change DESC
               ) AS product_rank
        FROM calculated_analysis
    )
    SELECT product_name,
           category,
           promotional_events,
           total_before,
           total_after,
           quantity_change,
           percentage_change,
           revenue_before,
           revenue_after,
           revenue_change,
           average_base_price,
           product_rank
    FROM ranked_products
    WHERE product_rank <= 2
    ORDER BY category, product_rank;

```